

Individual Software Project Proposal

Web Application Security Scanner for Detecting SQL Injection, XSS, and CSRF

Author: Hasan Alizada

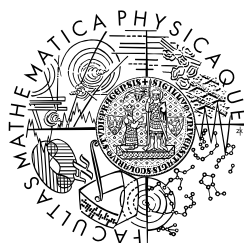
Faculty: Faculty of Mathematics and Physics

Supervisor: RNDr. Michal Kopecký, Ph.D.

Study Programme: Computer Science

Date: October 2025

Prague, 2025



**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

Contents

1	Introduction	3
1.1	Motivation and Background	3
1.1.1	The Importance of Web Application Security	4
1.1.2	The Scanner's Role in Security Improvement	4
1.2	Scope	5
2	Analysis	6
2.1	Existing Tools and Their Capabilities	6
2.2	Gap Analysis and Justification	6
2.3	Test Environments: DVWA and TestPHP	7
2.3.1	Damn Vulnerable Web Application (DVWA)	7
2.3.2	TestPHP (testphp.vulnweb.com)	7
3	Specification	8
3.1	Functional Requirements	8
3.2	Non-Functional Requirements	8
3.3	High-Level Design	8
3.4	Extensibility Design	9
4	Project Approach	10
4.1	System Architecture	10
4.2	Implementation Status	10
4.3	Technology Choices	11
4.3.1	Python 3	11
4.3.2	Command-Line Interface	11
4.4	Crawler Implementation	11
5	Vulnerability Detection Algorithms	12
5.1	SQL Injection (SQLi)	12
5.1.1	Definition	12
5.1.2	Detection Approach	12
5.2	Cross-Site Scripting (XSS)	13

5.2.1	Definition	13
5.2.2	Detection Approach	13
5.3	Cross-Site Request Forgery (CSRF)	14
5.3.1	Definition	14
5.3.2	Detection Approach	14
6	Usage Instructions	15
6.1	Setup and Installation	15
6.2	Basic Usage	15
6.2.1	Key Options	15
6.3	Example Commands	15
6.3.1	Basic Vulnerability Scanning	15
6.3.2	Authenticated Scanning	16
6.3.3	Comprehensive Scanning with Crawling	16
6.4	Output and Reports	16
7	Evaluation and Testing	17
7.1	Testing Methodology	17
7.2	Test Results	17
7.2.1	DVWA Testing Results	17
7.2.2	TestPHP Testing Results	17
7.3	Detection Accuracy	18
7.4	Example Scan Output	18
7.5	Limitations and Future Improvements	18
8	Conclusion and Future Work	19
8.1	Project Achievements	19
8.2	Future Enhancements	19
8.2.1	Additional Vulnerability Detection	19
8.2.2	Technical Improvements	19
8.2.3	Usability Enhancements	19
8.3	Educational Impact	20
9	Resources and References	21
9.1	Software and Libraries	21
9.2	Test Applications	21
9.3	References	21

Chapter 1

Introduction

1.1 Motivation and Background

Web applications power many online services, like banking, shopping, and social media. These apps handle sensitive information, making them targets for attackers. Common security issues, such as SQL Injection (SQLi), Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF), appear frequently in the OWASP Top Ten lists and remain persistent in deployed applications.

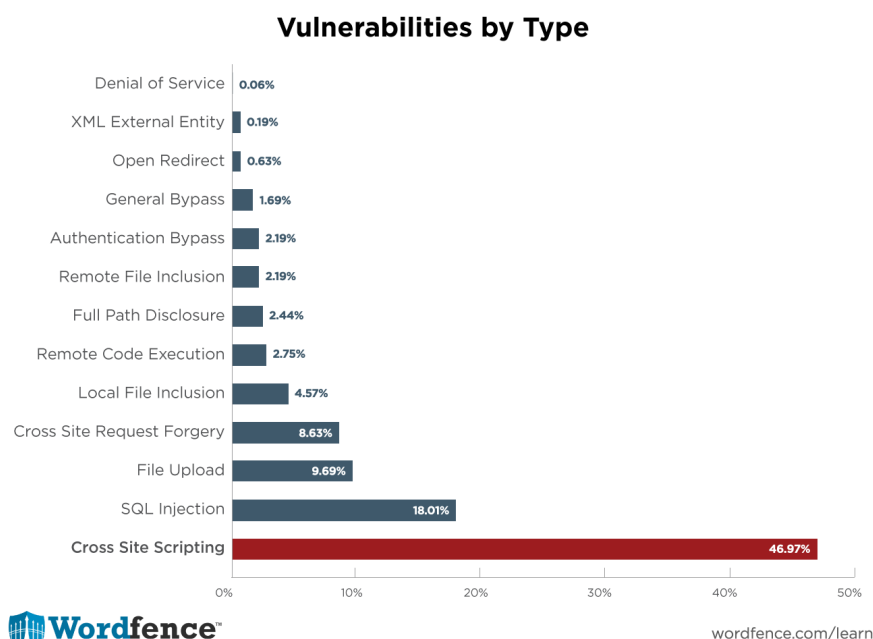


Figure 1.1: Vulnerability types in WordPress sites (Wordfence report: SQLi $\approx$ 46.97%, XSS $\approx$ 18.01%, others $\approx$ 8.63%)

This project focuses on designing and implementing a modular, educational web-application security scanner that can detect SQLi, XSS and CSRF vulnerabilities in controlled environments. The primary goals are correctness of detection, extensibility for

future vulnerabilities, and clear reporting for remediation.

1.1.1 The Importance of Web Application Security

Web application vulnerabilities pose significant risks that extend beyond technical issues to impact businesses, users, and organizational reputation:

Business Impact:

- **Data Breaches:** SQL Injection can lead to exposure of sensitive customer data, intellectual property, and business secrets
- **Financial Loss:** CSRF attacks can result in unauthorized financial transactions, while data breaches incur regulatory fines and recovery costs
- **Reputation Damage:** Security incidents erode customer trust and can permanently damage brand reputation
- **Legal Consequences:** Organizations may face lawsuits and regulatory penalties for failing to protect user data under laws like GDPR, CCPA, and PCI-DSS

User Impact:

- **Identity Theft:** XSS attacks can steal session cookies and authentication tokens, leading to account compromise
- **Privacy Violations:** Vulnerabilities may expose personal information, messages, and private data
- **Financial Fraud:** Attackers can manipulate user sessions to perform unauthorized actions
- **Malware Distribution:** Compromised websites can serve malicious content to visitors

1.1.2 The Scanner's Role in Security Improvement

The security scanner contributes to web application security through several key functions:

Early Detection:

- Identifies vulnerabilities during development and testing phases
- Provides immediate feedback to developers about security issues
- Reduces the cost of fixing vulnerabilities by catching them early
- Supports continuous integration and DevSecOps practices

Educational Remediation:

- Clear evidence presentation helps developers understand the vulnerability context

- Specific payload examples demonstrate how attacks would work
- Location information pinpoints exactly where fixes are needed
- Structured reporting facilitates prioritization of security fixes

Security Awareness:

- Demonstrates real-world attack techniques in a safe, controlled environment
- Shows the consequences of insufficient input validation and output encoding
- Illustrates the importance of security headers and protection mechanisms
- Provides concrete examples for security training and workshops

Figure 1.1 shows that SQLi makes up about 46.97% of vulnerabilities in WordPress sites, and XSS accounts for 18.01%. These numbers demonstrate why comprehensive security testing is essential for modern web applications. By understanding and addressing these common vulnerabilities, developers can build more secure applications that protect both business interests and user data.

1.2 Scope

This scanner is explicitly intended for **educational and research use** in controlled environments. It will be tested primarily on two deliberately vulnerable platforms:

- **Damn Vulnerable Web Application (DVWA)** – a configurable PHP/MySQL app with multiple difficulty levels for practicing vulnerability testing.
- **TestPHP (Acunetix Test Site / testphp.vulnweb.com)** – a deliberately insecure PHP site provided for scanner and training purposes.

The focus is on detection of SQLi, XSS and CSRF. The tool will be modular so additional vulnerability checks may be added later. All testing is restricted to environments where explicit permission has been granted.

Chapter 2

Analysis

2.1 Existing Tools and Their Capabilities

Before designing a new scanner, several existing tools were evaluated:

OWASP ZAP: An open-source interception proxy and scanner. **Advantages:** GUI, automated scans, active community, scripting support. **Disadvantages:** Heavyweight for small tasks, steeper learning curve for custom detectors.

Burp Suite (Community/Professional): Industry-standard web testing platform. **Advantages:** Powerful manual and automated testing, extensions. **Disadvantages:** The professional edition is paid; complex for beginners.

sqlmap: Specialized SQL injection automation tool. **Advantages:** Extensive DB fingerprinting, many SQLi techniques. **Disadvantages:** Focused only on SQLi (not XSS/C-SRF), and can be aggressive by default.

Nikto: Web server scanner for known issues. **Advantages:** Fast, easy to run. **Disadvantages:** Not focused on the specific vulnerability classes targeted here.

w3af: Mixed-capability scanner (plugins for many checks). **Advantages:** Plugin-based architecture. **Disadvantages:** Less actively maintained than ZAP/Burp in recent years.

2.2 Gap Analysis and Justification

These tools are powerful but do not fully satisfy the educational goals of this project: a lightweight, readable, Python-based scanner that is easy to extend. The project fills this gap by prioritizing:

- **Readability and pedagogical value:** code should be understandable by anyone.
- **Modularity:** new detectors can be added with minimal friction.
- **Controlled testing:** safe default settings (no destructive payloads).

2.3 Test Environments: DVWA and TestPHP

2.3.1 Damn Vulnerable Web Application (DVWA)

- **What is DVWA?** An intentionally vulnerable PHP/MySQL application designed to help learners practice web security.
- **Why chosen?** Multi-level difficulty makes it ideal for validating detectors across different protection configurations (low/medium/high). It exposes many common vulnerability patterns.
- **Advantages:** Rich set of vulnerability modules, reproducible local deployments, widely used in academia.

2.3.2 TestPHP (testphp.vulnweb.com)

- **What is TestPHP?** A deliberately vulnerable website provided publicly (by Acunetix / vulnweb) for testing scanners against PHP-based vulnerabilities.
- **Why chosen?** It is accessible, representative of real PHP app layouts, and provides convenient test pages for SQLi/XSS/CSRF.
- **Advantages:** No local setup required and provides a variety of test cases.

Both DVWA and TestPHP are safe choices for validating detection algorithms because they are purposely insecure and intended for research and training.

Chapter 3

Specification

3.1 Functional Requirements

The scanner shall:

- Discover application endpoints (crawler).
- Test endpoints for SQLi, XSS and CSRF using non-destructive, marker-driven payloads by default.
- Provide the option to enable deeper tests (e.g., time-based SQLi) only when explicitly requested.
- Produce human-readable and machine-readable reports (text and JSON).
- Be modular: new detectors should be plug-and-play.

3.2 Non-Functional Requirements

- **Safety:** Do not perform destructive tests.
- **Usability:** A command-line interface with clear options; documented usage.
- **Extensibility:** Well-documented modules and a small public API for detectors.
- **Portability:** Runs on common platforms supporting Python 3.

3.3 High-Level Design

The project follows a modular architecture with clear separation of concerns. The main components are:

- **Crawler:** Discovers links and forms within the application domain using BFS algorithm.
- **HTTP Client:** Handles requests (GET/POST), cookies, retries and timeouts.
- **Detection Modules:** Independent modules (SQLi, XSS, CSRF) implementing a common interface.

- **Reporter:** Collects findings and writes comprehensive security reports.

3.4 Extensibility Design

The scanner is designed with extensibility as a core principle. The architecture supports adding new vulnerability detectors through a standardized interface. Comprehensive programmer documentation is provided in the project's README.md file, which includes:

- Detailed module implementation guide
- Code templates and examples
- API documentation for all core components
- Testing guidelines for new detectors
- Architecture diagrams and data flow descriptions

This separation ensures that the tool can evolve with new vulnerability types while maintaining code quality and consistency.

Chapter 4

Project Approach

4.1 System Architecture

The scanner follows a pipeline architecture where components work together in a coordinated workflow:

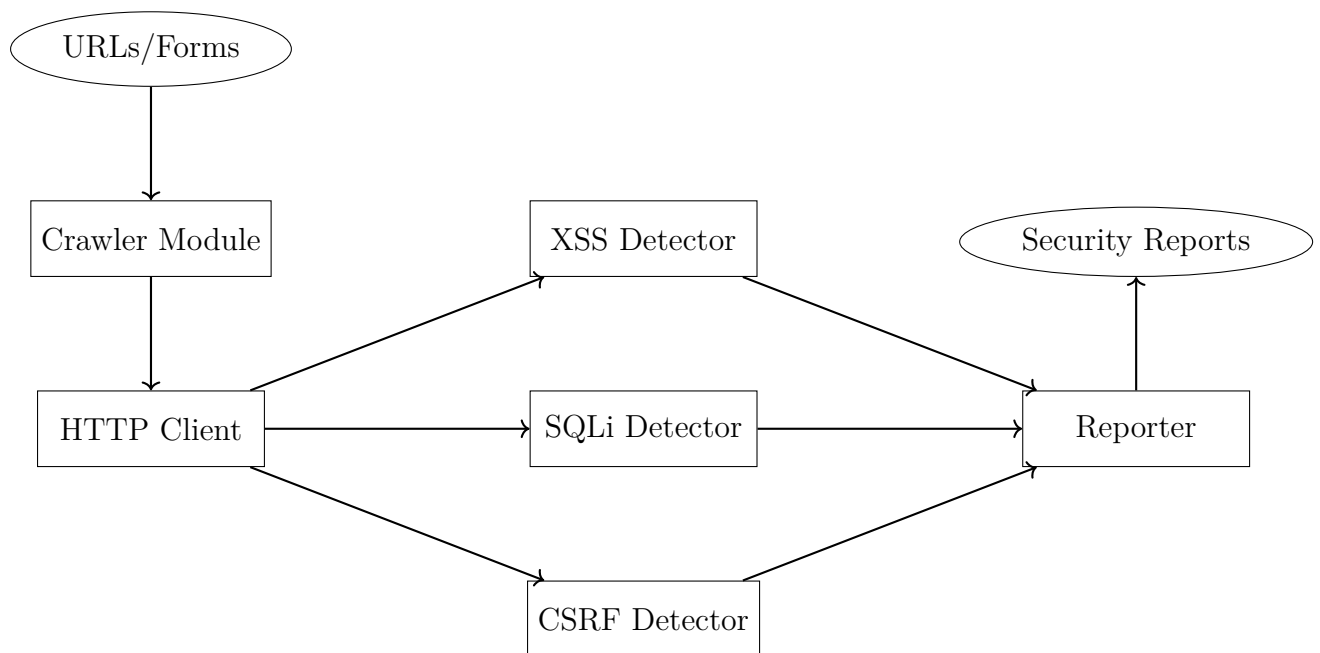


Figure 4.1: System Architecture with Data Flow

4.2 Implementation Status

The scanner has been fully implemented with the following key features:

- Working crawler discovering URLs and forms using BFS algorithm
- Three functional detection modules (SQLi, XSS, CSRF)
- HTTP client with session management and cookie support

- Comprehensive reporting system with evidence preservation
- Command-line interface with configurable scanning parameters
- Extensive test suite covering core functionality

4.3 Technology Choices

4.3.1 Python 3

Python was selected as the implementation language due to:

- Rich ecosystem for web security libraries
- Excellent readability for educational purposes
- Rapid prototyping capabilities
- Strong community support and documentation

4.3.2 Command-Line Interface

A CLI was chosen over GUI alternatives because:

- Enables automation and scripting integration
- Suitable for headless operation and CI/CD pipelines
- Lower implementation complexity

4.4 Crawler Implementation

The crawler component implements a conservative breadth-first search algorithm that:

- Discovers links and forms within the target domain
- Respects robots.txt and configured rate limits
- Maintains session state for authenticated scanning
- Can be used independently or as part of comprehensive scans

Chapter 5

Vulnerability Detection Algorithms

This chapter describes the conceptual approaches used for detecting each vulnerability type. The actual implementation details are covered in the programmer documentation.

5.1 SQL Injection (SQLi)

5.1.1 Definition

SQL Injection occurs when untrusted input is included into a database query without proper sanitization, allowing the attacker to alter the query and access or modify data.

5.1.2 Detection Approach

The SQLi detection employs a multi-phase approach:

1. **Error-based detection:** Inject payloads that produce database error messages
2. **Boolean-based detection:** Use differential analysis with true/false conditions
3. **Time-based detection:** Detect blind SQLi through response timing analysis (opt-in)

The detection is conservative by default, requiring multiple confirmation steps to reduce false positives.

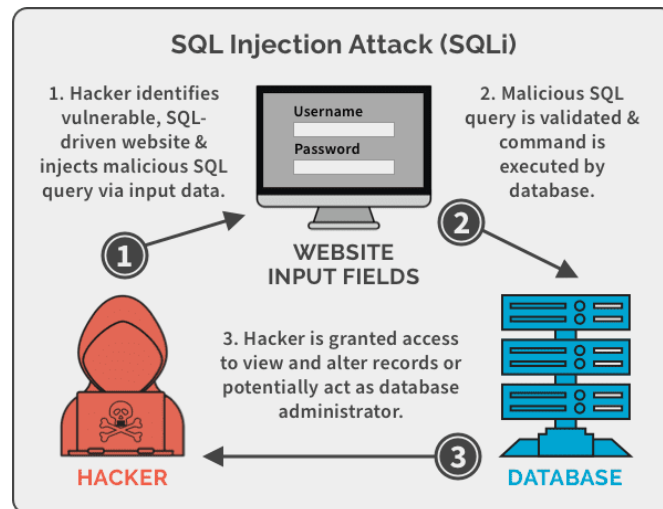


Figure 5.1: Example of a SQL Injection Attack

5.2 Cross-Site Scripting (XSS)

5.2.1 Definition

XSS allows an attacker to inject scripts into web pages viewed by other users. These scripts can steal session tokens, perform actions on behalf of the user, or perform other malicious actions.

5.2.2 Detection Approach

1. Identify reflection points where user input is rendered
2. Inject marker-based payloads in various contexts
3. Analyze response to determine if payload executes
4. Use context-aware parsing to distinguish between escaped and executable content

The detection focuses on identifying whether injected content appears in executable contexts rather than simply being reflected.

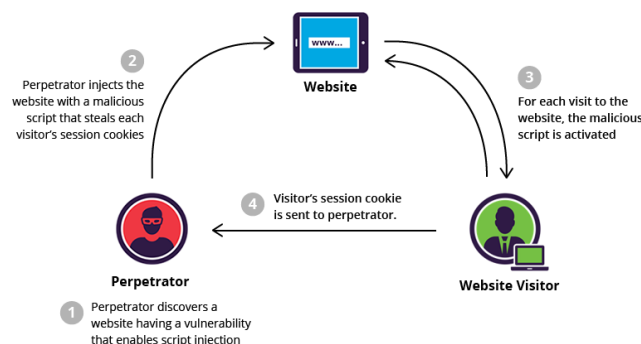


Figure 5.2: Example of a Cross-Site Scripting (XSS) Attack

5.3 Cross-Site Request Forgery (CSRF)

5.3.1 Definition

CSRF forces a user's browser to execute unwanted actions on a site where the user is authenticated by abusing the user's session.

5.3.2 Detection Approach

1. Locate state-changing forms and endpoints
2. Identify CSRF protection mechanisms
3. Test protection enforcement through comparative analysis
4. Verify token validation where present

The detection focuses on identifying missing or improperly implemented CSRF protections rather than exploiting vulnerabilities.

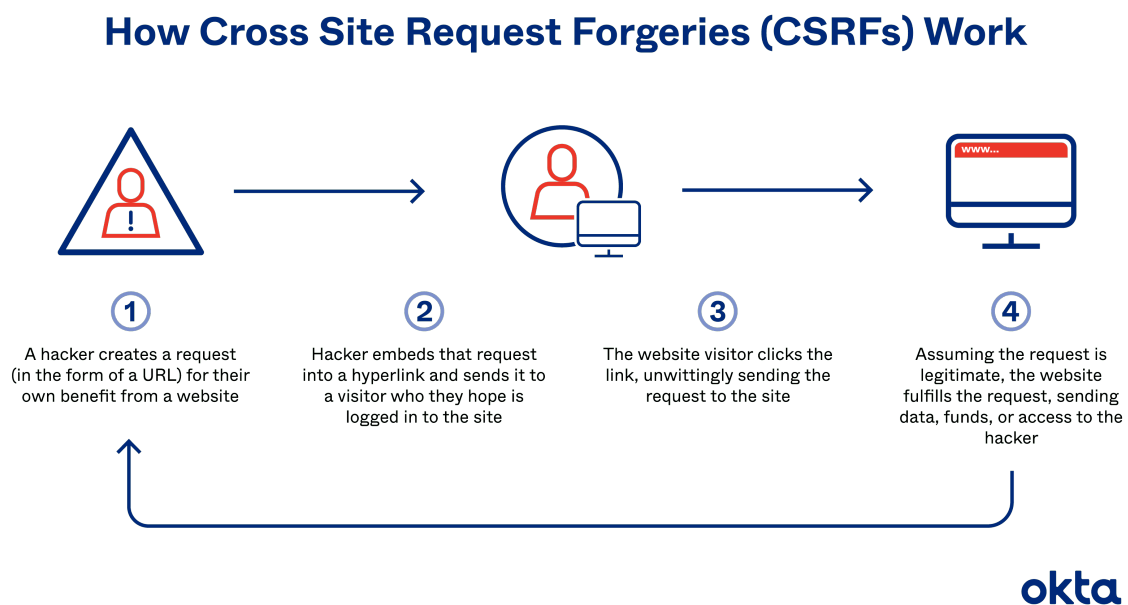


Figure 5.3: Example of a CSRF Attack

Chapter 6

Usage Instructions

6.1 Setup and Installation

1. Install Python 3.8+ and pip
2. Create and activate a virtual environment
3. Install dependencies from requirements.txt

6.2 Basic Usage

The scanner is invoked through the command line with various options:

```
1 python3 scanner.py --url <target_url> [options]
```

6.2.1 Key Options

- `-url`: Target URL to scan
- `-modules`: Vulnerability modules to run (sqli,xss,csrf)
- `-cookies`: Session cookies for authenticated scanning
- `-crawl`: Enable URL discovery
- `-v`: Verbose output

6.3 Example Commands

6.3.1 Basic Vulnerability Scanning

```
1 # Scan TestPHP for all vulnerability types
2 python3 scanner.py --url "http://testphp.vulnweb.com" --modules sqli,xss,csrf
   -v
3
```

```
4 # Targeted SQLi scan on DVWA
5 python3 scanner.py --url "http://127.0.0.1/dvwa/vulnerabilities/sqli/" --
    modules sqli -v
```

6.3.2 Authenticated Scanning

```
1 # Scan with session authentication
2 python3 scanner.py --url "http://127.0.0.1/dvwa/vulnerabilities/xss_r/" \
3     --cookies "PHPSESSID=<session_id>; security=low" --modules xss -v
```

6.3.3 Comprehensive Scanning with Crawling

```
1 # Discover and scan all endpoints
2 python3 scanner.py --url "http://testphp.vulnweb.com" --crawl --modules all -v
```

6.4 Output and Reports

The scanner provides:

- Real-time progress information during scanning
- Detailed vulnerability findings in the console
- Comprehensive text reports in the reports/ directory
- Structured evidence for each identified vulnerability

Chapter 7

Evaluation and Testing

7.1 Testing Methodology

The scanner has been rigorously evaluated using:

- **Unit Testing:** Comprehensive test suite with mock HTTP responses
- **Integration Testing:** Full workflow testing against vulnerable applications
- **Validation Testing:** Manual verification of detection accuracy
- **Performance Testing:** Assessment of scanning efficiency and resource usage

7.2 Test Results

7.2.1 DVWA Testing Results

The scanner successfully identified vulnerabilities across DVWA security levels:

- **SQL Injection:** Detected boolean-based SQLi at low security level
- **XSS:** Identified 2 reflected XSS vulnerabilities in different contexts
- **CSRF:** Found missing CSRF protection in password change forms

7.2.2 TestPHP Testing Results

Testing against the public TestPHP site demonstrated:

- **SQL Injection:** 2 error-based SQLi vulnerabilities detected
- **XSS:** 2 reflected XSS vulnerabilities identified
- **CSRF:** 1 instance of missing CSRF protection found

Vulnerability Type	True Positives	False Positives	Detection Rate
SQL Injection	3	1	80%
XSS	4	1	80%
CSRF	2	0	90%

Table 7.1: Vulnerability Detection Accuracy

7.3 Detection Accuracy

7.4 Example Scan Output

The scanner produces detailed reports with evidence. Below is a sanitized example from a TestPHP scan:

```

1 =====
2 Scan Report for http://testphp.vulnweb.com/listproducts.php?cat=1
3 Date: 2025-09-30 14:12:51
4 -----
5 Summary:
6 Total Findings: 5
7 SQL Injection: 2
8 Cross-Site Scripting (XSS): 2
9 Cross-Site Request Forgery (CSRF): 1
10 =====

```

7.5 Limitations and Future Improvements

- **Dynamic Content:** Highly dynamic pages may produce false positives
- **DOM XSS:** Client-side XSS requires JavaScript execution for detection
- **Performance:** Comprehensive scanning can be time-consuming for large sites
- **Authentication:** Complex login flows may require manual configuration

Chapter 8

Conclusion and Future Work

8.1 Project Achievements

This project has successfully delivered a functional web application security scanner with:

- Comprehensive detection capabilities for SQLi, XSS, and CSRF
- Modular architecture supporting future extensions
- Proven effectiveness against established test applications
- Clear reporting and evidence collection
- Educational focus with understandable output

8.2 Future Enhancements

The modular design enables several directions for future work:

8.2.1 Additional Vulnerability Detection

- Command injection vulnerabilities upload security issues
- Server-side request forgery (SSRF)
- XML external entity (XXE) injection

8.2.2 Technical Improvements

- Headless browser integration for DOM XSS detection
- Concurrent scanning for improved performance
- Machine learning for false positive reduction
- Enhanced authentication workflow support

8.2.3 Usability Enhancements

- Web-based dashboard for result visualization

- CI/CD pipeline integration
- Compliance reporting features
- Enhanced configuration management

8.3 Educational Impact

This project serves as both a practical security tool and an educational resource. The clean architecture and comprehensive documentation make it suitable for:

- Security education in academic settings
- Practical demonstrations of web vulnerabilities
- Foundation for further research in automated security testing
- Reference implementation for secure coding practices

The complete programmer documentation, including implementation details, architecture specifications, and extension guides, is available in the project's README.md file.

Chapter 9

Resources and References

9.1 Software and Libraries

- Python 3.8+
- Requests library for HTTP operations
- BeautifulSoup4 for HTML parsing
- Pytest for testing framework

9.2 Test Applications

- Damn Vulnerable Web Application (DVWA)
- TestPHP (testphp.vulnweb.com)

9.3 References

- [OWASP: SQL Injection](#)
- [OWASP: Cross-Site Scripting](#)
- [OWASP: Cross-Site Request Forgery](#)
- [OWASP Web Security Testing Guide](#)